illustrated in Fig. 2, the process begins when a target is provided to the reconnaissance agent. The agent operates in a self-iterating loop, generating reconnaissance commands to gather information from the target and analyzing the results of these commands until the best efforts have been made. Once the reconnaissance loop concludes, the agent summarizes its findings and stores them in a database.

The reconnaissance agent adheres to a general workflow defined with expert knowledge to perform the reconnaissance task. It determines specific procedures or tools to use with the help of external knowledge supported by the RAG framework. To achieve our desired workflow, we carefully design the system messages and prompts for the reconnaissance agent, implementing the following techniques to overcome the challenges mentioned in §2.2.

Reconnaissance System Message (Simplified)

Role-play

You're an excellent cybersecurity penetration tester assistant. Guide the tester …

Chain-of-Thought

Use Nmap to identify exposed ports, then use relevant tools in Nmap to analyze these ports on the target host …

RAG

You should use your query tool to learn about available reconnaissance tools …

Structured Output

You should always respond in valid JSON format with the following fields: {FORMAT SPEC.} …

Role-playing has proven effective in bypassing the safety policies enforced by the LLM [11]. Thus, we ask the reconnaissance agent to act as a penetration tester assistant to validate its reconnaissance behaviors.

We use Chain-of-Thought (CoT) to break down complex tasks into several sub-tasks and construct an effective reconnaissance workflow to reduce hallucination. Since the reconnaissance workflow involves a self-iterating loop, it is important to specify a stop condition to avoid the agent getting into an infinite loop. Using CoT effectively enforces the stop condition by specifying the tasks to complete before stopping.

Retrieval-Augmented Generation (RAG) allows the reconnaissance agent to retrieve relevant information from a database containing documentation of various reconnaissance tools, enabling it to use up-to-date tools for effective information collection. For example, it can use web application fingerprinting tools with open-source fingerprinting databases like ObserverWard [1] to aid in reconnaissance. Furthermore, RAG allows the reconnaissance agent to store collected environmental information in a database for later use, addressing the short-term memory issue.

The reconnaissance agent analyzes previous execution results and generates the next command to execute in each communication. To enforce adherence to the penetration testing pipeline and ensure a smooth transition to subsequent steps, we use structured output, asking the reconnaissance agent to respond using a specified format.

After the reconnaissance agent determines that it should stop the reconnaissance loop, it summarizes the reconnaissance results and stores them in a database to make the short-term reconnaissance memory persistent.

3.3 Search Agent

The search agent takes target services and applications as input and stores relevant attack knowledge into databases as output. As illustrated in Fig. 3, the search agent performs two rounds of hierarchical online search for relevant information. In the first round, it searches and analyzes the results to extract potential attack surfaces relevant to the target. In the subsequent round, it uses the identified potential attack surfaces as a guide to search and analyze procedure-level attack knowledge. The potential attack surfaces and procedure-level attack knowledge are stored in two separate databases for future use.

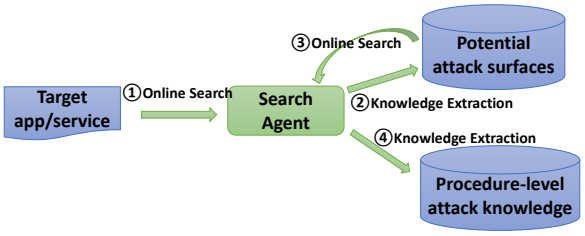


Figure 3: Search agent workflow

The online search module is customizable and extensible. We have implemented several search functions, including general searches on Google, vulnerability-specific searches on databases like Snyk [40] and AVD [9], and searches in exploit code repositories such as GitHub and ExploitDB. In our hierarchical search workflow, we use Google and vulnerability database searches to identify potential attack surfaces in the first round and then employ Google and code repository searches to find exploit implementation details in the second round.

After each round of online searches, the search agent analyzes the results. However, indexing and storing information from raw search results is inefficient. Therefore, we leverage RAG-based question-answering to extract key information from the raw search results and use the extracted knowledge to build a more relevant and concise database. As elucidated in Fig. 4, given the analysis prompt, the RAG framework will first retrieve relevant segments of information from the search results. Then, it sends the analysis prompt with the retrieved information as context to LLM as the question, and the LLM will analyze the information in the context to help answer the queries in the analysis prompt and generate a comprehensive summary for the search results containing the key information we are looking for. Finally, the summaries of individual documents are gathered to build a potential attack surface or exploit database in Fig. 3.

For the first round of searching for potential attack surfaces, we use the following prompt to extract knowledge from individual search results. Specifically, we ask for relevancy and key information about vulnerabilities, such as CVE numbers, as well as other keywords or URLs that can lead to more detailed information. We

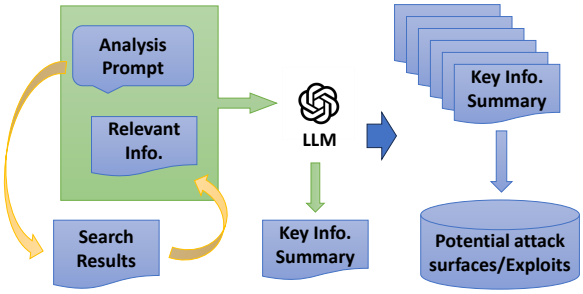


Figure 4: RAG workflow for search result summarization. The yellow arrows denote the retrieval process, and the green arrows denote the generation process.

also ask the search agent to output the analysis results in a structured format for subsequent processing.

Potential Attack Surface Analysis Prompt (Simplified)

RAG & CoT

Generate a concise summary of the document to answer the following questions:
1) Does this document describe vulnerabilities targeting a particular service or app; if so, what is the relevant service/app version?
2) Provide information that can be used to search for the exploit of the vulnerabilities …

Structured Output

You should always respond in valid JSON format with the following fields: {FORMAT SPEC.} …
For example, the response looks like this: {OUTPUT FORMAT EXAMPLE}

Similarly, for the second round of searching for procedure-level exploit details, the search agent analyzes individual search results using RAG and CoT. First, it checks whether the repository contains a relevant exploit. Then, it extracts key information such as applicable service or application versions and prerequisites for running the exploit. While the first round of analysis mainly focuses on the LLM's text summarization capability, the second round relies on the LLM's code analysis capability to determine whether the code functions as an exploit and the dependencies required to execute it.

After the penetration testing knowledge is extracted by the search agent, it is stored in a hierarchical tree structure as shown in Fig. 5. The hierarchical tree-structured penetration testing knowledge base allows efficient searching and systematic management of penetration testing knowledge.

3.4 Planning Agent

The planning agent takes the detected services and applications from the reconnaissance agent as input and generates an exploitation plan as output. As shown in Fig. 1, the planning agent leverages RAG and the pentesting knowledge base (Fig. 5) to first generate a list of potential attack surfaces relevant to services and applications. Then, the planning agent follows a similar process to generate a list of exploits.

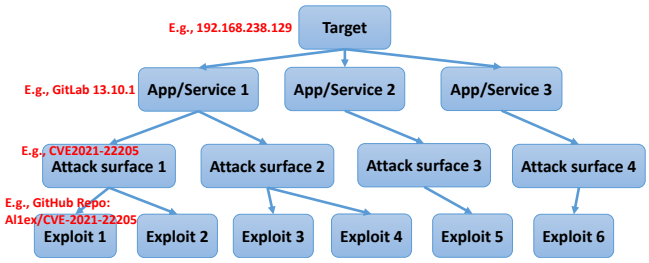


Figure 5: Hierarchical pentesting knowledge database

The planning agent uses the service or application as a key to find the relevant database for potential attack surfaces and retrieves these from the database according to the version of the service or application and the types of vulnerabilities. The planning agent makes suggestions for attack surfaces based on the application version and categorizes attack surfaces by vulnerability types.

The planning agent then uses the attack surface to find the relevant database for exploits and retrieves exploit details from the database according to the service or application version and exploit effects (e.g., remote code execution, authentication bypass). The planning agent then makes suggestions for exploits based on the application version and categorizes the exploits by exploit effects.

3.5 Execution Agent

The execution agent takes the details of the exploit as input and attempts to execute the exploits on the target automatically, ultimately generating an exploitation summary as output. The execution agent follows the order suggested by the planning agent. As illustrated in Fig. 6, each exploit execution can be divided into two stages: the preparation stage and the exploitation stage.

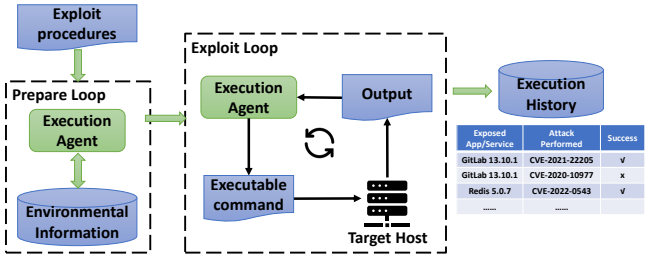


Figure 6: Execution agent workflow

During the preparation stage, the execution agent analyzes the details of the exploit to determine the requirements for successful execution, such as the parameters needed. It then queries the database containing environmental information to obtain the necessary information. The following system message guides the execution agent in fulfilling its duty. To bypass the safety mechanisms that may prevent the process while the execution agent tries to execute the exploit, we use role-playing to make it act as a cybersecurity penetration tester.

To effectively find the information needed to execute the exploit, we employ the CoT technique to guide the execution agent to first identify all the parameters and then determine the information needed for each parameter. During this analysis, the execution agent retrieves relevant information from the exploit details using

RAG to provide the context. Finally, the execution agent outputs the required information in a structured JSON format, allowing it to proceed to the subsequent step of requesting information from the environmental information database.

Execution Preparation System Message (Simplified)

Role-play

You're an excellent cybersecurity pentesting assistant …

CoT

Analyze the exploit to answer the following questions:
1) What parameters for executing this exploit?
2) What information do you need to execute the exploit?

RAG

Use your query tool to learn the details about …

Structured Output

You should always respond in valid JSON format with the following fields: {FORMAT SPEC.} …

After obtaining a list of needed information, the execution agent queries the environmental information database to fill in the information. After receiving the response containing the requested information, the execution agent's system message is updated as follows to transition to the exploitation stage.

Execution Exploit System Message (Simplified)

Your next task is to provide step by step guide for executing the exploit and debugging the errors encountered …

RAG

You should use the query tool to learn the code and README of the exploit to figure out how to properly execute it. You also use …

Self-reflection

When the results indicate an error, you should analyze the error and try to fix it …

Structured Output

You should always respond in valid JSON format with the following fields: {FORMAT SPEC.} …

During the exploitation stage, the execution agent uses RAG to obtain details of the code execution, breaks down the execution plan, and generates a step-by-step execution guide. Similar to the reconnaissance agent, the execution agent engages in iterative loops to execute the exploit.

When errors are encountered during exploit execution, proper error handling is required. To guide the execution agent in debugging errors, we employ the self-reflection technique. The execution agent analyzes and fixes errors based on the code and error message while concurrently documenting the error history for future reference to avoid repeating the error. This iterative process ensures continual refinement and optimization of our automated pentesting system.

4 Evaluation

In this section, we present the benchmark established for evaluating automated penetration testing frameworks and discuss the evaluation results. We address the following research questions (RQs) in our evaluation:

RQ1. Effectiveness. What's the success rate of finishing the whole penetration testing process automatically?

RQ2. Completion level. What's the completion level of individual penetration testing stages that can be automatically finished?

RQ3. Efficiency. How much time and API cost are needed for PentestAgent to complete a penetration testing task?

4.1 Evaluation Setup

4.1.1 Benchmark Dataset. The benchmark dataset should be easily accessible and include a diverse set of tasks with varying difficulty levels to evaluate the automated penetration testing framework. Accessibility is essential for a good benchmark; otherwise, it prevents the whole community from using it. The tasks in the benchmark should involve exploiting various vulnerabilities targeting different services and applications to mimic real-world penetration testing scenarios. More importantly, the tasks should have appropriate difficulty labels to reflect how well the system under test can handle tasks of different difficulty levels, helping researchers identify the strengths and weaknesses of the system.

Several platforms can serve as the dataset of the benchmark, such as HackTheBox [17], OWASP Benchmark [33], VulnHub [46], and VulHub [45]. OWASP Benchmark and VulnHub contain thousands of target testing environments, covering a wide range of real-world penetration testing scenarios. However, setting up these environments for testing requires significant human effort. Furthermore, they do not provide a difficulty level reference for their test cases, necessitating manual effort to determine the difficulty level for each test case.

Finally, we chose VulHub and HackTheBox as our benchmark dataset. VulHub provides an open-source collection of over a hundred pre-built vulnerable Docker environments, which has been widely recognized and utilized in penetration testing practices. The container-based platform supports infrastructure as code (IaC), making it easy to set up the testing environments. Besides, Docker containers provide sufficient isolation for penetration testing. Moreover, most vulnerable environments in VulHub are constructed to reproduce a particular Common Vulnerabilities and Exposures (CVE) [28]. Each vulnerable environment is associated with a CVE number, which allows us to use metrics associated with CVE numbers to learn about the properties of each vulnerable environment. Specifically, we learn about the difficulty of vulnerability exploits through the Common Vulnerability Scoring System (CVSS)[10] and learn about how realistic the vulnerable environment is via the Exploit Prediction Scoring System (EPSS)[32]. We elaborate on how we construct the benchmark dataset in §B.1 in the appendix.

As a result, we compiled a benchmark comprising 67 penetration testing targets, spanning 32 CWE (Common Weakness Enumeration) categories as shown in Fig.13 in the appendix. These vulnerabilities cover eight security risks in OWASP Top 10 vulnerability [34]. Within our benchmark, there are 50 targets with easy exploitability difficulty, 11 with medium exploitability difficulty, and

6 with hard exploitability difficulty. In addition, we incorporated 11 Capture The Flag (CTF) challenges from HackTheBox to simulate more challenging and realistic scenarios. These challenges are used in Section 4.5 for a practicality study and in Section 4.6 for the comparative evaluation with PentestGPT. This diverse and realistic collection of vulnerable environments ensures a comprehensive assessment.

4.1.2 Metric. To answer our research question, we design metrics to evaluate the effectiveness and efficiency of PentestAgent. These metrics are essential for assessing the performance of the automated penetration testing framework.

We measure the effectiveness of PentestAgent by determining whether all three stages of penetration testing are completed successfully and automatically. We define successful completion as follows: given a target IP, PentestAgent can automatically perform a functional exploit on the vulnerable environments. For the HackTheBox targets, our focus is on obtaining initial access to the target host. In this context, a successful exploit is one that grants access to the target system.

Some penetration tests may be partially successful and require human assistance. However, failure in a previous penetration testing stage will affect the subsequent stages. To better understand the effectiveness of each component in PentestAgent, we measure the completion level at the stage level. This involves assessing the penetration testing stages that can be completed, assuming the preceding stages have been successful. The completion criteria for each stage are defined as follows. The information gathering stage is considered complete if the target application is successfully identified by PentestAgent. The vulnerability analysis stage is marked as complete when PentestAgent identifies functional exploits based on the target application. We manually verify whether the discovered exploits are effective. The exploitation stage is completed if PentestAgent can automatically and successfully execute the exploit. This stage-level evaluation provides a granular understanding of PentestAgent 's autonomy and effectiveness in progressing through the penetration testing process with minimal human assistance.

Furthermore, we measure the efficiency of PentestAgent using the time taken and the API cost incurred to complete penetration tests. The time metric evaluates the duration required for PentestAgent to complete an entire penetration test cycle, from initial reconnaissance to exploit execution. the API cost metric quantifies the computational resources consumed by the framework during the testing process. These metrics provide insights into the system's resource consumption and operational speed, which are critical for practical deployment and scalability.

4.1.3 Environment setup. The simulated vulnerable applications are hosted on a virtual machine with 2 CPU cores and 8 GB RAM, running Ubuntu 22.04 LTS. To avoid interference with the testing process, we have disabled all services that require listening on ports, such as SSH. The attacker machine is also hosted on a virtual machine with 16 CPU cores and 16 GB RAM, running Kali Linux 2024.1. The attacker machine includes all the pre-installed tools available in Kali Linux, with no additional tools installed. The victim machine and the attacker machine maintain network connectivity via NAT. The vulnerable containers on the victim machine are created with the network parameter set to the victim machine's IP, allowing the attacker machine to directly access the vulnerable environments hosted in the victim machine's containers. This setup ensures the attacker can simulate real-world network conditions when attempting to exploit the vulnerabilities.

We evaluate our framework using a mix of commercial and open-source LLMs. Table 2 summarizes their key properties.

4.2 Effectiveness of the Entire Framework

We investigate the effectiveness of PentestAgent by its success rates of completing the penetration testing process. Fig. 7 shows the success rates of exploiting vulnerabilities categorized by difficulty levels and overall performance across different models. The GPT-4 model demonstrated a 74.2% overall success rate in completing automated penetration testing tasks, outperforming the GPT-3.5 model, which achieved a 60.6% success rate. Both models consistently achieved success rates above 60%, affirming the effectiveness of PentestAgent in establishing an automated penetration testing pipeline.

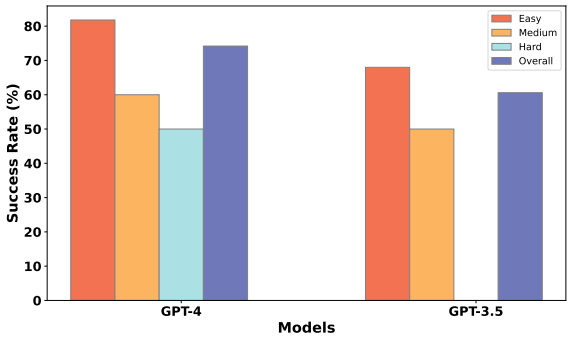


Figure 7: Success rate on penetration testing tasks

While the GPT-4 model showed a higher overall success rate compared to GPT-3.5, the difference between their performances was not substantial. This suggests that our framework does not rely heavily on LLMs' general knowledge and capabilities alone.

Notably, the GPT-3.5 model struggled particularly with hard penetration testing tasks, achieving no success in the hardest category. This disparity likely stems from the inherent differences in context window size and learned knowledge between the models, impacting their ability to handle complex reasoning required for challenging tasks.

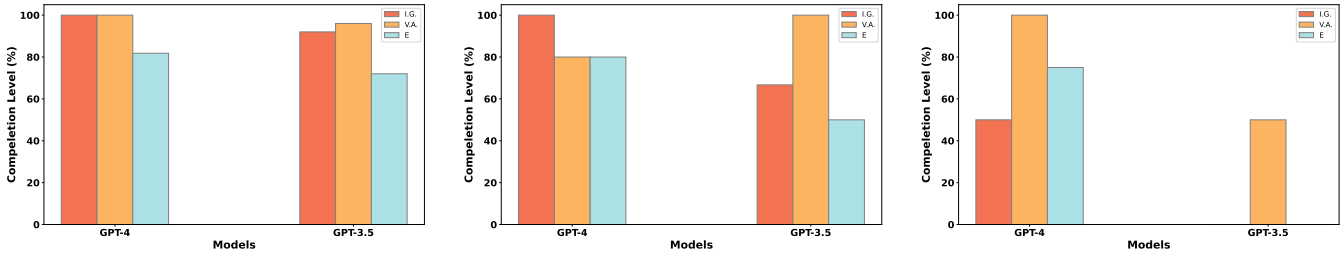
4.3 Completion level of Penetration Testing Stages

To further evaluate PentestAgent, we analyze its performance across individual penetration testing stages. Fig. 8 presents the completion rates for intelligence gathering, vulnerability analysis, and exploitation across different difficulty levels and LLM backbones.

GPT-4 demonstrated robust performance across all stages and difficulty levels, effectively handling a range of penetration testing tasks. In easy tasks, it achieved full completion in both intelligence gathering and vulnerability analysis, with an 81.8% completion rate in exploitation. For medium-difficulty tasks, GPT-4 maintained high completion rates across all stages, with a minor drop in vulnerability

Table 2: Summary of LLM models used in our evaluation. Input/output costs are based on pricing at the time of testing.

Model	Context Window	Knowledge Cutoff	Input Cost	Output Cost
GPT-3.5-turbo-0125	16,385 tokens	Sep 2021	$0.50/1M tokens	$1.50/1M tokens
GPT-4o	128,000 tokens	Oct 2023	$5.00/1M tokens	$15.00/1M tokens
o1-mini	128,000 tokens	Oct 2023	$1.10/1M tokens	$4.40/1M tokens
Llama 3.1-8B-Instruct	128,000 tokens	Dec 2023	Open-source	Open-source



(a) Easy tasks **(b) Medium tasks** **(c) Hard tasks**

Figure 8: Completion level of penetration testing stages on different difficulty of tasks. I.G. denotes the intelligence gathering stage, V.A. denotes the vulnerability analysis stage, and E denotes the exploitation stage.

analysis. However, in hard tasks, performance declined, particularly in intelligence gathering (50%), suggesting limitations in handling complex reconnaissance tasks that require advanced reasoning and adaptive strategies.

In contrast, GPT-3.5 exhibited more variability across difficulty levels. It performed well in easy tasks, with 92% completion in intelligence gathering and 96% in vulnerability analysis, though its exploitation stage completion rate (72%) was slightly lower. For medium tasks, while maintaining a 100% completion rate in vulnerability analysis, its performance declined in intelligence gathering (66.7%) and exploitation (50%), indicating challenges in navigating complex reconnaissance and execution scenarios. Notably, in hard tasks, GPT-3.5 struggled significantly, failing to complete both intelligence gathering and exploitation, highlighting its limitations in reasoning and contextual understanding required for advanced penetration testing.

Overall, both models effectively automate significant portions of penetration testing, but GPT-4 consistently outperforms GPT-3.5, particularly in more complex scenarios. These results emphasize the importance of advanced reasoning capabilities and enhanced reconnaissance strategies in achieving higher success rates in automated penetration testing.

4.4 Ablation Study

To assess how different LLM backbones influence PentestAgent's performance, we performed an ablation study on a subset of VulHub targets. Due to hardware limitations, specifically insufficient GPU resources to efficiently run the Llama 3.1 model on the full dataset, we selected a smaller subset comprising six easy, five medium, and two hard targets. Fig. 9a and Fig. 9b summarize the completion levels and overhead results, respectively. The results show that while all models perform consistently in vulnerability analysis (with 100% completion), differences emerge in the other stages. For the intelligence gathering stage, GPT-4 and o1-mini achieve higher completion levels compared to GPT-3.5 and Llama 3.1-8B-Instruct, suggesting that certain models are more effective in integrating context and managing complex reasoning. In the exploitation stage, o1-mini outperforms the others, indicating its strength in executing detailed attack procedures, whereas GPT-3.5 and Llama 3.1-8B-Instruct fall behind.

Overhead measurements further highlight trade-offs between processing time and cost. Although GPT-3.5 is faster in intelligence gathering and is the most cost-effective, its lower exploitation performance suggests a potential compromise in handling complex tasks. In contrast, while o1-mini delivers the best exploitation completion, it incurs longer processing times during vulnerability analysis. The Llama 3.1-8B-Instruct model, despite having no cost, suffers from significant time overhead and lower exploitation performance, which may limit its practical use.

4.5 Practicality Study

To evaluate PentestAgent's effectiveness in realistic settings, we deployed it to solve HackTheBox challenges. Unlike standardized benchmarks, these challenges simulate dynamic, real-world penetration testing tasks by presenting diverse vulnerabilities across various CWEs. In this study, we selected 11 HackTheBox machines, nine labeled as easy, one as medium, and one as hard, to provide a broad assessment of the framework's practical utility.

Fig. 10 shows the completion level and overhead of PentestAgent on HackTheBox challenges. Table 3 in Appendix C further details PentestAgent's performance on these challenges by reporting the number of completed testing stages. While PentestAgent successfully exploited six machines, others like *Pilgrimage* achieved only partial completion, with only the vulnerability analysis stage being successful.

Overall, these results indicate that PentestAgent can address a range of real-world scenarios. However, the variability in stage completion underscores the need for further improvements to achieve more consistent, full-stage automation. We discuss the failed cases in detail in Section 4.7.